

Analysis: Burger Optimization

Burger Optimization: Analysis

Small dataset

This dataset can be solved using complete search. Since $K \leq 8$, there are only at most $8! = 40320$ different orderings of ingredients.

Therefore, we can try all $K!$ different orderings of ingredients and find which ordering of ingredients has the minimum sum of squared differences between each ingredient's optimal and actual distance-to-bun values. Since we can calculate the sum of the squared differences between each ingredient's optimal and actual distance-to-bun values for one particular ordering of ingredients in $O(K)$ time, this solution runs in $O(K! \times K)$ time.

Large dataset

We need an observation to solve this dataset. Suppose we have two ingredients A and B to be placed at two positions X and Y. Let ingredients A and B have optimal distance-to-bun values of D_A and D_B , respectively, and let positions X and Y have actual distance-to-bun values of D_X and D_Y , respectively. Suppose $D_A \leq D_B$ and $D_X \leq D_Y$. We claim that it is no less optimal to put ingredient A at X and to put ingredient B at Y.

Why? Let us assume that $D_B = D_A + K_1$ and $D_Y = D_X + K_2$ for some non-negative integers K_1 and K_2 . Therefore, the cost of putting ingredient A at X and putting ingredient B at Y is $(D_A - D_X)^2 + (D_B - D_Y)^2$, and the cost of putting ingredient A at Y and putting ingredient B at X is $(D_A - D_Y)^2 + (D_B - D_X)^2$.

We can derive that

$$\begin{aligned} & [(D_A - D_X)^2] + [(D_B - D_Y)^2] \\ &= [D_A^2 - 2D_AD_X + D_X^2] + [D_B^2 - 2D_BD_Y + D_Y^2] \\ &= [D_A^2 - 2D_AD_X + D_X^2] + [D_B^2 - 2(D_A + K_1)(D_X + K_2) + D_Y^2] \\ &= [D_A^2 - 2D_AD_X + D_X^2] + [D_B^2 - 2D_AD_X - 2D_AK_2 - 2K_1D_X - 2K_1K_2 + D_Y^2] \\ &= [D_A^2 - 2D_AD_X - 2D_AK_2 + D_Y^2] + [D_B^2 - 2D_AD_X - 2K_1D_X + D_X^2] - [2K_1K_2] \\ &= [D_A^2 - 2D_A(D_X + K_2) + D_Y^2] + [D_B^2 - 2(D_A + K_1)D_X + D_X^2] - [2K_1K_2] \\ &= [D_A^2 - 2D_AD_Y + D_Y^2] + [D_B^2 - 2D_BD_X + D_X^2] - [2K_1K_2] \\ &= [(D_A - D_Y)^2] + [(D_B - D_X)^2] - [2K_1K_2] \end{aligned}$$

Therefore, $(D_A - D_X)^2 + (D_B - D_Y)^2 \leq (D_A - D_Y)^2 + (D_B - D_X)^2$, since $2K_1K_2$ is a non-negative integer. Therefore, we have proved that it is no less optimal to put ingredient A at X and to put ingredient B at Y. In other words, we want to put an ingredient which has a smaller optimal distance-to-bun value in a position which has a smaller actual distance-to-bun value.

Based on the above observation, the solution to this problem becomes simple. Let D' be the list of actual distance-to-bun values for the ingredients sorted in non-decreasing order (e.g. if K is 7, then $D' = \{0, 0, 1, 1, 2, 2, 3\}$), and let D be the list of optimal distance-to-bun values for the

ingredients sorted in non-decreasing order. Using the observation above, it is optimal to put the i -th ingredient on the list of D to a position which has an actual distance-to-bun value of $D'[i]$. Therefore, the answer is simply $\sum (D'[i] - D[i])^2$.

We can sort the ingredients in $O(K \log(K))$ time and calculate the sum of the squared differences between each ingredient's optimal and actual distance-to-bun values in $O(K)$ time. Therefore, this solution runs in $O(K \log(K))$ time.

(For a similar approach, you might enjoy reading about the [rearrangement inequality](#)).

Analysis: Centrists

Centrists: Analysis

Small dataset

In the Small dataset, the candidates' names can only contain the letters A, B, and C, so we can effectively disregard the rest of the alphabet. That is, since there are no Ds anywhere in any of the names, for example, we do not care whether A comes before or after D in an ordering of the alphabet. Only the relative order of A, B, and C matters, and there are only $3! = 6$ such orders, so we can check each one of them and see how the three names get sorted in each case. Then, we can report YES for any name that appeared in the middle under at least one ordering, and NO for any name that did not.

Large dataset

The English alphabet has $26!$ possible orderings; this is close to 4×10^{26} , which is far too many for us to check individually. We will take another approach. Let us examine the first letter of each name. If all three of those letters are the same, then they cannot influence the order in which the names get sorted, so we can move on to looking at the second letters, and so on. Eventually, we will find an index at which the letters are not all the same; there are two ways in which this can happen.

The first of these — all three are different — is the easiest to deal with. Let L_i denote the letter at that index in the i -th name. Then, if we choose an alphabet ordering in which L_i falls somewhere between the other two letters, the i -th name will be in the middle. So the answer is YES for all three names.

Otherwise, two of the letters at that index are the same, and one is different. Call the shared letter L_s and the different letter L_d , and call the name with L_d at that index N_d . We can already see that N_d can never end up in the middle. If $L_s < L_d$ (where $<$ means "comes before in the alphabet ordering"), then the other two names will come before N_d . (Other differences that come later in the names do not influence this.) Otherwise, N_d will come before the other two names.

What about the order of the other two names? Let us scan through them, starting just after our aforementioned index, looking for the earliest index at which the letters of these names are different. Call the differing letters at that index L_1 and L_2 , and call the names containing those letters (respectively) at that index N_1 and N_2 . Let us consider the various possible identities of L_1 and L_2 .

Suppose that $L_1 = L_s$ and $L_2 = L_d$. Then, if $L_s < L_d$, our name order will be N_1, N_2, N_d . Otherwise, it will be N_d, N_2, N_1 . So N_1 cannot be in the middle, but N_2 can. A similar situation holds for $L_1 = L_d$ and $L_2 = L_s$.

Otherwise, regardless of the identities of L_1 and L_2 , either of N_1 and N_2 could be in the middle:

- Suppose that L_1 (without loss of generality) $= L_d$. We can choose an ordering in which L_d comes first to get the order N_d, N_1, N_2 , or we can choose an ordering in which $L_s < L_d < L_2$

to get the order N_1, N_2, N_d .

- Suppose that L_1 (without loss of generality) $= L_s$. We can choose an ordering in which L_s comes first to get the order N_1, N_2, N_d , or we can choose an ordering in which $L_d < L_s < L_2$ to get the order N_d, N_1, N_2 .
- Otherwise, we can choose an ordering that begins with L_d (which puts N_d first), and has L_1 and L_2 in the order in which we want N_1 and N_2 to appear.

Analysis: CEO Search

CEO Search: Analysis

Small dataset

In the Small dataset, there are at most 10 existing employees, so one viable approach is as follows. We will start by supposing that the new CEO has the minimum experience level possible: one level higher than the highest level among all existing employees. Then, we can check all possible assignments of employees to potential managers of higher levels than those employees. (Throughout this analysis, for convenience, we will use "manager" to mean "direct manager"). If any of the assignments is valid — that is, nobody is managing too many employees — then we have solved the problem. Otherwise, we try again with a CEO of one level higher, and so on. This process is bounded, since the answer cannot possibly be larger than 11; the existing employees are all level 10 or lower, so a level 11 CEO could manage all of them personally.

At first, it may seem that the search space is too large. When we have one employee of each level between 1 and 10, for example, and we try to add a new CEO of level 11, there are 10! possible sets of assignments. (The level 1 employee has 10 potential managers, the level 2 employee has 9 potential managers, and so on.) But the cases like this one with large search spaces are also easy to solve; note that any assignment is valid in our example, since it is impossible to give a manager too many employees. On the other hand, when we have a case like five level 0s, four level 1s, and one level 2, and we try to add a new CEO of level 3, it turns out that no set of assignments is valid... but there are so few possible sets of assignments that it is easy to check them all and move on to trying a new CEO of level 4.

Large dataset

Suppose that we have chosen a new CEO of some level (which may or may not turn out to work), and we have started trying to assign managers in some way; not everyone necessarily has a manager yet. Suppose that an employee A of level L_A is managing an employee B of level L_B , and there is some other employee C of level L_C (not managed by A) such that $L_A > L_C > L_B$. Then we can safely have A manage C instead of B. If employee C previously had a manager D, that manager is of a high enough level to manage employee B, so we can have D manage B and we are no worse off than before. If employee C did not yet have a manager, then we are better off than before. The same logic holds if employee B stands for an empty space rather than an employee.

This observation suggests a top-down approach to the problem. We can work from the highest-level employees to the lowest-level ones, and apply a simple rule: each employee should manage as many other employees as they can, and those employees should be of the highest levels available among all remaining unmanaged employees.

We can simplify this approach by thinking in terms of levels instead of employees. We will start at the top level, which has the new CEO of level X ; the new CEO creates X management "slots". Then, let us consider the next highest level, which has N_L employees of level E_L . If $N_L > X$, then our plan does not work and we need a CEO with a higher level. Otherwise, we have used up N_L of our X slots, but the employees from this level have added $N_L \times E_L$ more slots. We can continue in this way, abandoning a plan whenever we do not have enough slots to manage all of the employees at a given level.

For the Large dataset, the CEO level may be as large as the largest possible N_i value plus 1, so we may not have time to check all of them. We can speed up the process by [binary searching](#) on the level of our new CEO, with an inclusive lower bound equal to $E_L + 1$, and an inclusive upper bound equal to the lower bound or the maximum N_i allowed by the problem, whichever is greater. (If there are 10^{12} level 0 employees, for example, the answer could be as large as 10^{12} .) If we try a possible value and find that it works, we can make that value our new upper bound; if a value does not work, we can set the lower bound to that value plus 1. Each search takes $O(L)$ time, and the binary search adds another factor of $\log(\max(N_i))$, so the running time is $O(L \times \log(\max(N_i)))$.

This is fast enough to solve the Large dataset, but we can do even better by cutting out the binary search entirely! We can proceed through the levels as before, but without including the new CEO or trying to guess their level. Whenever we encounter employees with no possible managers, we add them to a count and then move on. (This is definitely the case for every employee of level E_L , and it may be true of employees at other levels as well.) The final answer is then either that count (since all of those unmanaged employees must be personally managed by the new CEO), or $E_L + 1$, whichever is greater.

It is also possible to work from the bottom up instead, using the same idea, or to apply [Hall's theorem](#) to get a closed-form expression of the solution; we leave those as exercises. In any case, our running time is now $O(L)$.

Analysis: Tricky Trios

Tricky Trios: Analysis

Playing the game

Before we can approach either of the datasets, we need to figure out an optimal strategy for the game. One important insight is that we should not begin a round by flipping over a known card, unless we know where all three cards in a trio are (in which case we can flip them all to remove that trio).

To see why this is, suppose that we begin by flipping the one or two known cards with a certain number. Then, when we move on to flipping unknown cards, we either find the one or two remaining cards we need, or we fail and learn the identity of only one more card. But if we instead begin by flipping an unknown card, we have exactly the same chance of encountering the cards needed to remove the aforementioned trio, in which case we can end the round by flipping the known ones. We also have some other advantages: we have a chance of removing any of the other trios, and if we fail to do that, we will learn the identities of two more cards. So, beginning with an unknown card is strictly better than beginning with a known card; it leaves more options open and gets us more valuable information.

What about the parts of a round other than the beginning? There is one situation in which we have some flexibility: if the unknown card revealed on our first flip matches one known card, there is no harm in flipping over that known card as our second action before flipping over an unknown card as our third action. But it is no worse to flip over an unknown card as our second action instead, since we need to find the last member of that trio either way.

So, the following simple rule is optimal: only flip known cards when they are the last ones needed to remove a trio.

Small dataset

This is an unusual dataset for Code Jam: there are only five possible test cases, and three of them are given away as samples, so we can hardcode those in and only focus on solving the $N = 3$ and $N = 4$ cases.

Notice that whenever we flip an unknown card, we have no basis for choosing any particular unknown card. Without loss of generality, we can choose to reveal them from left to right. With this in mind, one tempting approach is to repeatedly simulate playing the game, each time choosing a random left-to-right order in which to deal with the cards, and then take the average number of rounds. The tight requirement of an absolute or relative error of 10^{-6} makes this challenging, though; even millions of simulations might not get us close enough! We can let a simulation run for quite some time and get answers before starting the 4-minute submission window, but even optimized code may not be fast enough, especially in an interpreted language, for example.

Instead of simulating random left-to-right orders, can we enumerate them all, play through each one, and then take the average number of rounds? For our $N = 4$ case, there are a total of $(12 \text{ choose } 3) \times (9 \text{ choose } 3) \times (6 \text{ choose } 3) \times (3 \text{ choose } 3) = 369600$ orders, which is a tractably small amount. Although it is not necessary, we can cut this down by another factor of $N!$ by noting that the numbers on the cards are essentially interchangeable; an order like

111222333444 will yield the same result as an order like 222333444111. This drops the number of cases to 15400 for $N = 4$, and 1401400 for $N = 5$. Beyond that point, though, there are just too many orders to consider individually.

Large dataset

At the start of each round of the game, for each trio that we have not already removed, we know the locations of three, two, one, or zero of the cards, depending on what we have flipped over on earlier rounds. Let us classify trios accordingly as Three-Known, Two-Known, One-Known, or Zero-Known. Then we can consider the number of trios of each type — (K_3, K_2, K_1, K_0) — as a *state*, and think of a round as a transition from one state to another state. We begin the game at $(0, 0, 0, N)$, and we want to get to $(0, 0, 0, 0)$ as efficiently as possible, but our path will depend on how lucky we get when flipping over unknown cards.

Let us use $f(\text{state})$ to denote the question "how many rounds will it take, on average, to finish the game when starting from this state?" To calculate $f(\text{state})$, we must consider all the states in which the round can end, and the probabilities of reaching each of those destination states. Then $f(\text{state})$ is a sum of the $f()$ s for those destination states, weighted by the probability of reaching each of them.

We can simplify this model at the outset by noting that if we play according to our strategy above, K_3 will never be larger than 1, and if it is 1, we should spend the next round to immediately remove the Three-Known trio. That is, $f(K_3, K_2, K_1, K_0) = 1 + f(K_3 - 1, K_2, K_1, K_0)$. So we can remove the K_3 term and work with states of the form (K_2, K_1, K_0) , and add in an extra round as needed to reflect removing a Three-Known trio.

For example, suppose that we start a round in a state $(1, 1, 1)$; this means that there are six unknown cards remaining, with one from a Two-Known trio, two from a One-Known trio, and three from a Zero-Known trio. (Remember that there are other known cards around as well — for instance, the other two cards from the Two-Known trio — but we are focusing on the unknown cards.) Let us consider one possible scenario: our first draw turns out to be from the One-Known trio, and then our second draw turns out to be from the Two-Known trio, which ends the round. This converts our Two-Known trio into a Three-Known trio and our One-Known trio into a Two-Known trio. We should immediately spend the next round to remove the Three-Known trio.

The probability of the above happening is $2/6$ (the odds of drawing one of the unknown cards from the One-Known trio) $\times 1/5$ (the conditional odds of drawing the unknown card from the Two-Known trio). So, when we write the expression for $f(1, 1, 1)$, it should include the term $2/6 \times 1/5 \times (2 + f(1, 0, 1))$. The 2 represents the current round plus the extra round spent removing the Three-Known trio. Note that $(1, 0, 1)$ reflects that we have converted a One-Known trio into a Two-Known trio and lost another Two-Known trio.

The above illustrates the calculation of just one term, and our solution needs to be able to handle any state; the more general version of the term above would be $((2 \times K_1) / (K_2 + 2 \times K_1 + 3 \times K_0)) \times (K_2 / (K_2 + 2 \times K_1 - 1 + 3 \times K_0)) \times (2 + f(K_2, K_1 - 1, K_0))$. Setting up terms like this is the heart of the problem, and there are various possible pitfalls, e.g.:

- If our first flip on a round reveals a card from a Zero-Known trio, our second flip might reveal a card from any of the following: a Two-Known trio, a One-Known trio, the same Zero-Known trio, or a different Zero-Known trio. It is important to distinguish between the latter two possibilities.
- We must avoid considering terms that represent impossible situations; we cannot draw a card from a different Zero-Known trio if there is only one Zero-Known trio in our starting state.

Once we have all this nailed down, we need a way to avoid computing $f()$ for the same state more than once. We can store each result and then look it up again later if we need it for a future calculation, instead of doing redundant work. This is [memoization](#), a form of [dynamic programming](#). This optimization allows the entire Large dataset to be solved in seconds.